

MECHATRONICS AND IOT

ASSIGNMENT - 10

Smart Irrigation and Water Management System

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

Smart Irrigation and Water Management System

1. Project Overview

The Smart Irrigation and Water Management System is an IoT-based agricultural system designed to automate the irrigation process according to the moisture condition of the soil. The system continuously monitors soil moisture and automatically controls the water pump when the soil becomes dry.

An ESP8266 NodeMCU is used as the main controller. The soil moisture sensor measures the moisture level in the soil and sends the reading to the ESP8266. Based on a predefined moisture threshold, the controller switches the water pump ON or OFF using a relay module.

The system helps reduce unnecessary water consumption, minimizes manual intervention, and provides efficient irrigation for agricultural applications. The system can also be connected to a cloud platform for remote monitoring of soil-moisture data and irrigation status.

Main components

- ESP32
- Soil Moisture Sensor
- Relay Module
- Mini Water Pump
- DHT11 Sensor
- Breadboard
- Jumper Wires

2. Project Statement

Agriculture requires an adequate and timely supply of water for healthy crop growth. Traditional irrigation methods often depend on manual monitoring and fixed watering schedules, which can result in over-irrigation, under-irrigation, and unnecessary water wastage.

The lack of continuous monitoring of soil moisture makes it difficult to determine the exact amount of water required by crops. This can increase water consumption and may negatively affect plant growth.

The objective of this project is to design and develop a Smart Irrigation and Water Management System that automatically monitors soil moisture and controls irrigation according to the actual water requirements of the soil. The system uses a **soil moisture sensor and ESP8266 NodeMCU** to monitor soil conditions and automatically control a water pump through a relay.

The proposed system reduces manual intervention, improves water-use efficiency, and provides a reliable method for automated irrigation in agricultural applications. IoT connectivity can also be incorporated to enable remote monitoring of soil moisture and irrigation status through a cloud platform.

3. Proposed Solution

The proposed system uses a soil moisture sensor to determine the moisture level of the soil. The sensor is connected to an ESP8266 NodeMCU, which acts as the main control unit.

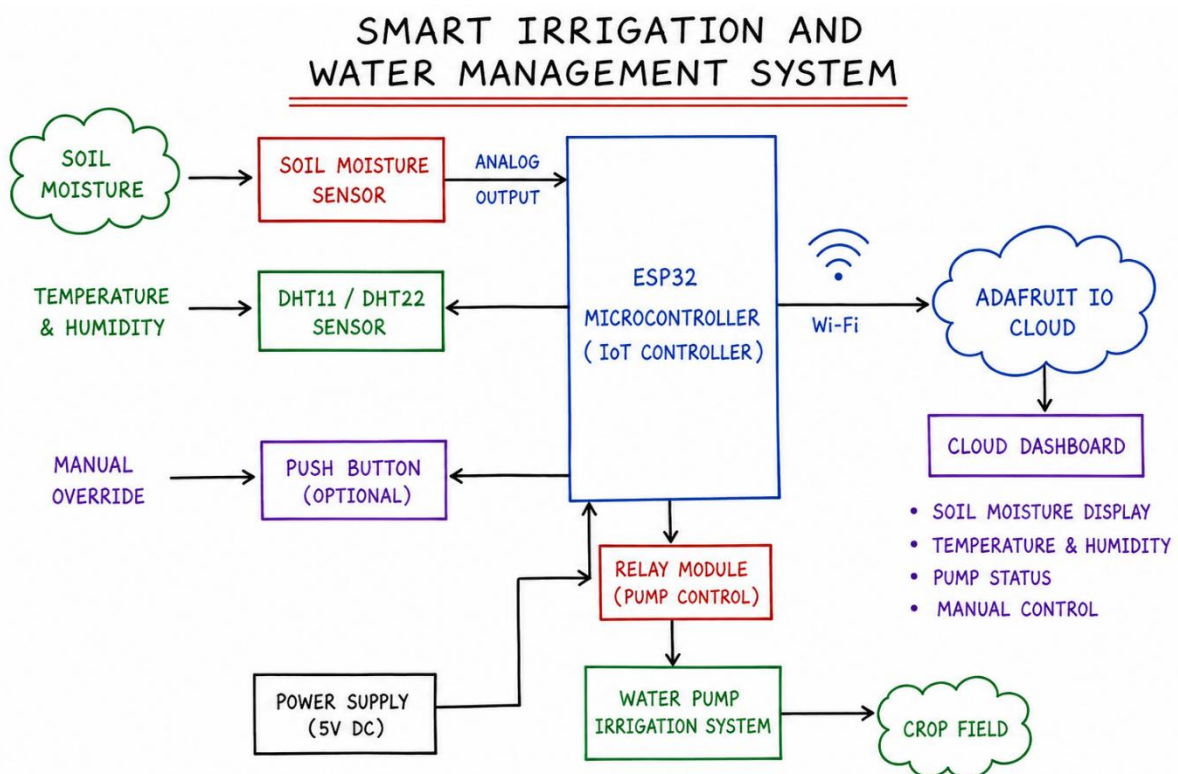
When the soil moisture level falls below the selected threshold, the ESP8266 activates a relay. The relay switches ON the water pump and supplies water to the plants.

When sufficient moisture is detected, the ESP8266 switches OFF the relay and stops the pump.

The basic operation is:

1. Soil moisture is measured using the soil moisture sensor.
2. The sensor sends the moisture reading to the ESP8266.
3. The ESP8266 compares the reading with a predefined threshold.
4. If the soil is dry, the relay is activated.
5. The water pump turns ON.
6. When sufficient moisture is reached, the relay is deactivated.
7. The water pump turns OFF.
8. Sensor data and pump status can be monitored remotely through the cloud platform.

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. SOIL MOISTURE SENSOR CONNECTION

Component	Pin	ESP32 (NodeMCU) Pin	Function
Soil Moisture Sensor	VCC	3.3V	Power Supply (3.3V)
	GND	GND	Ground
	AO	GPIO34 (ADC)	Analog Output to ESP32

2. DHT11 / DHT22 SENSOR CONNECTION

Component	Pin	ESP32 (NodeMCU) Pin	Function
DHT11 / DHT22 Sensor	VCC	3.3V	Power Supply (3.3V)
	GND	GND	Ground
	DATA	GPIO4	Data Signal to ESP32

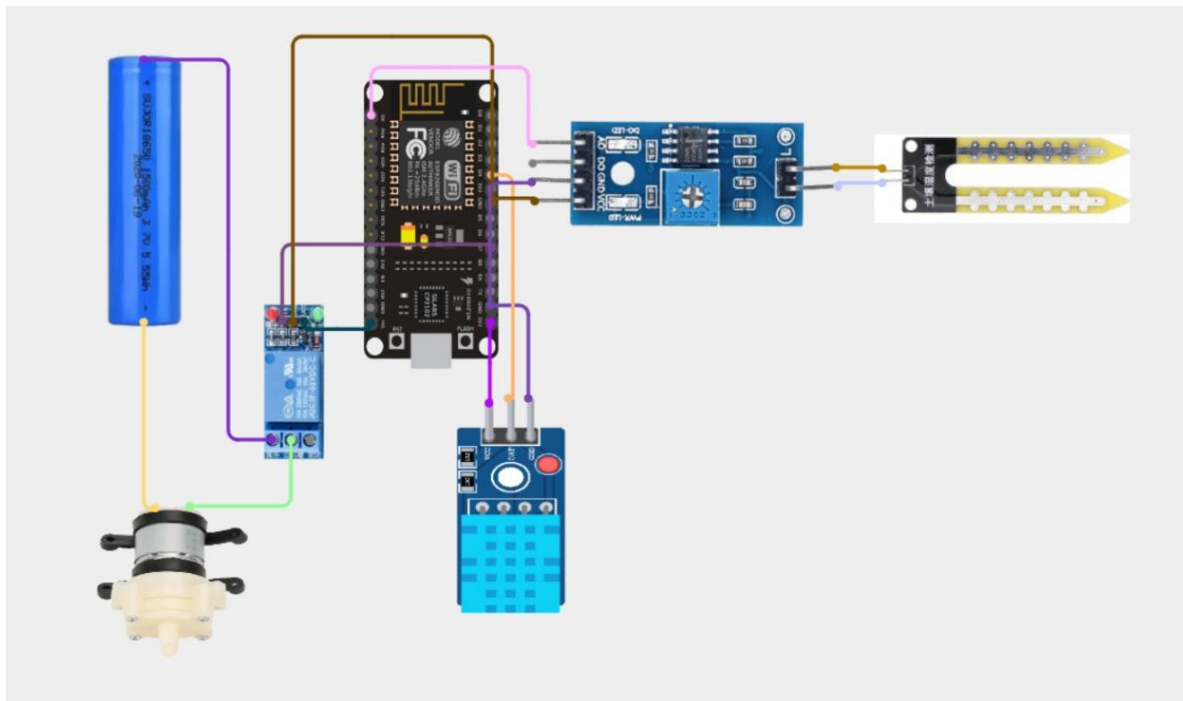
3. RELAY MODULE (PUMP CONTROL) CONNECTION

Component	Pin	ESP32 (NodeMCU) Pin	Other Connection	Function
Relay Module	VCC	VIN / 5V	5V DC Power Supply	Power Supply to Relay Module
	GND	GND	Common Ground	Ground
	IN	GPIO27	–	Control Signal from ESP32
Water Pump	+	External DC Supply (+)	Through Relay (NO)	Positive Supply to Pump
	–	External DC Supply (–)	Common Ground	Negative Supply to Pump

4. POWER SUPPLY

Component	Pin	Connection	Function
ESP32 (NodeMCU)	VIN	5V DC (USB / External 5V Supply)	Power Supply to ESP32
ESP32 (NodeMCU)	GND	Common Ground	Ground

6. Circuit Design:



7. Algorithm:

Start the System

- Power ON the ESP32.
- Initialize the soil moisture sensor, DHT11/DHT22 sensor, relay module, and Wi-Fi.

Read Soil Moisture

- The soil moisture sensor measures the moisture level of the soil.
- The ESP32 reads the analog value through its ADC pin.
- Convert the sensor value into a moisture percentage after calibration.

Read Temperature and Humidity

- The DHT11/DHT22 sensor measures the surrounding temperature and humidity.
- The ESP32 receives and processes these values.

Check Soil Condition

- Compare the measured soil moisture with predefined thresholds.
- If the soil is dry, irrigation is required.
- If the soil is sufficiently moist, irrigation is not required.

Control the Water Pump

- If soil moisture is below the dry threshold, the ESP32 activates the relay.
- The relay switches ON the water pump.
- Water is supplied to the crop field.
- When the soil reaches the required moisture level, the ESP32 switches the relay OFF.

Send Data to Adafruit IO

- The ESP32 connects to Adafruit IO through Wi-Fi.
- Soil moisture, temperature, humidity, and pump status are uploaded to the cloud dashboard.

Repeat the Process

- The ESP32 continuously reads the sensors and controls irrigation automatically.
-

8. Coding:

```
#include <WiFi.h>

#include "DHT.h"

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"


// =====

// Wi-Fi Details

// =====

#define WIFI_SSID "OnePlus Nord CE5 7C5C"

#define WIFI_PASSWORD "qkts5737"


// =====

// Adafruit IO Details

// =====

#define AIO_SERVER "io.adafruit.com"

#define AIO_SERVERPORT 1883


#define AIO_USERNAME "Sivasaravanan"

#define AIO_KEY "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"
```

```
// =====  
// Sensor and Relay Pins  
// =====  
#define SOIL_PIN 34  
#define RELAY_PIN 26  
#define DHT_PIN 4  
#define DHT_TYPE DHT11  
  
// =====  
// DHT Object  
// =====  
DHT dht(DHT_PIN, DHT_TYPE);  
  
// =====  
// Wi-Fi and MQTT  
// =====  
WiFiClient client;  
  
Adafruit_MQTT_Client mqtt(  
  &client,  
  AIO_SERVER,  
  AIO_SERVERPORT,  
  AIO_USERNAME,  
  AIO_KEY  
);  
  
// =====  
// Adafruit IO Feeds
```

```
// =====
```

```
Adafruit_MQTT_Publish soilFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/soil-moisture"  
);
```

```
Adafruit_MQTT_Publish temperatureFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/temperature"  
);
```

```
Adafruit_MQTT_Publish humidityFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/humidity"  
);
```

```
Adafruit_MQTT_Publish pumpFeed =
```

```
Adafruit_MQTT_Publish(  
    &mqtt,  
    AIO_USERNAME "/feeds/pump-status"  
);
```

```
// =====
```

```
// Soil Moisture Threshold
```

```
// =====
```

```
// Change this value after checking your sensor readings.
// Higher value = generally drier for many analog sensors.
int moistureThreshold = 2000;

// =====

// Connect ESP32 to Wi-Fi
// =====

void connectWiFi() {

    Serial.println();
    Serial.print("Connecting to Wi-Fi");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("Wi-Fi Connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}

// =====
```

```
// Connect to Adafruit IO
// =====

void MQTT_connect() {

    int8_t ret;

    // Already connected?
    if (mqtt.connected()) {
        return;
    }

    Serial.print("Connecting to Adafruit IO...");

    while ((ret = mqtt.connect()) != 0) {

        Serial.print("Connection failed: ");
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();

        Serial.println("Retrying in 5 seconds...");

        delay(5000);
    }

    Serial.println("Adafruit IO Connected!");
}
```

```
// =====  
// SETUP  
// =====  
  
void setup() {  
  
    Serial.begin(115200);  
  
    delay(1000);  
  
    Serial.println();  
    Serial.println("=====");  
    Serial.println(" ESP32 SMART IRRIGATION SYSTEM");  
    Serial.println("=====");  
  
    // Soil sensor  
    pinMode(SOIL_PIN, INPUT);  
  
    // Relay  
    pinMode(RELAY_PIN, OUTPUT);  
  
    // Relay OFF initially  
    // Most relay modules are ACTIVE LOW  
    digitalWrite(RELAY_PIN, HIGH);  
  
    // Start DHT11  
    dht.begin();  
  
    // Connect Wi-Fi
```

```
connectWiFi();

// Connect Adafruit IO
MQTT_connect();

Serial.println("System Ready!");
}

// =====
// LOOP
// =====

void loop() {

// Make sure MQTT is connected
MQTT_connect();

// =====
// Read Soil Moisture
// =====

int soilValue = analogRead(SOIL_PIN);

// =====
// Read DHT11
// =====

float temperature = dht.readTemperature();
```

```
float humidity = dht.readHumidity();

// =====

// Check DHT11 Reading

// =====

if (isnan(temperature) || isnan(humidity)) {

    Serial.println();
    Serial.println("ERROR: DHT11 reading failed!");

    delay(2000);

    return;
}

// =====

// Automatic Pump Control

// =====

bool pumpON = false;

if (soilValue > moistureThreshold) {

    // Soil is dry
    digitalWrite(RELAY_PIN, LOW);

    pumpON = true;
```

```
} else {

    // Soil is wet
    digitalWrite(RELAY_PIN, HIGH);

    pumpON = false;
}

// =====
// Serial Monitor
// =====

Serial.println();
Serial.println("=====");

Serial.print("Soil Moisture : ");
Serial.println(soilValue);

Serial.print("Temperature : ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidity   : ");
Serial.print(humidity);
Serial.println(" %");

Serial.print("Soil Status : ");

if (soilValue > moistureThreshold) {
```

```
    Serial.println("DRY");
} else {
    Serial.println("WET");
}

Serial.print("Pump Status  : ");

if (pumpON) {
    Serial.println("ON");
} else {
    Serial.println("OFF");
}

Serial.println("=====");

// =====
// Send Soil Moisture to Adafruit IO
// =====

Serial.println("Sending data to Adafruit IO...");

if (soilFeed.publish((int32_t)soilValue)) {

    Serial.println("✓ Soil moisture sent");

} else {

    Serial.println("X Soil moisture failed");
}
```

```
// =====
```

```
// Send Temperature
```

```
// =====
```

```
if (temperatureFeed.publish((double)temperature)) {
```

```
    Serial.println("✓ Temperature sent");
```

```
} else {
```

```
    Serial.println("X Temperature failed");
```

```
}
```

```
// =====
```

```
// Send Humidity
```

```
// =====
```

```
if (humidityFeed.publish((double)humidity)) {
```

```
    Serial.println("✓ Humidity sent");
```

```
} else {
```

```
    Serial.println("X Humidity failed");
```

```
}
```

```
// =====
```

```
// Send Pump Status
```

```
// =====
```

```
int32_t pumpStatus = pumpON ? 1 : 0;
```

```
if (pumpFeed.publish(pumpStatus)) {
```

```
    Serial.println("✓ Pump status sent");
```

```
} else {
```

```
    Serial.println("X Pump status failed");
```

```
}
```

```
Serial.println("Data transfer completed.");
```

```
Serial.println();
```

```
// =====
```

```
// Wait 5 Seconds
```

```
// =====
```

```
delay(5000);
```

```
}
```

9. System Working :



When the system is powered ON, the ESP32 initializes all connected sensors and establishes a Wi-Fi connection.

The soil moisture sensor continuously measures the moisture content of the soil. The ESP32 reads this value and converts it into a usable moisture percentage after calibration.

At the same time, the DHT11/DHT22 measures temperature and humidity. The water-level sensor checks the availability of water in the tank.

If the soil is sufficiently moist, the ESP32 keeps the pump OFF. This prevents unnecessary water consumption.

When the soil becomes dry and enough water is available in the tank, the ESP32 activates the relay. The relay switches ON the water pump and water is supplied to the crop.

As irrigation continues, soil moisture increases. When the moisture reaches the upper threshold, the ESP32 switches OFF the relay and stops the pump.

The sensor readings and pump status can be uploaded to an IoT dashboard. This allows the user to monitor the system remotely.

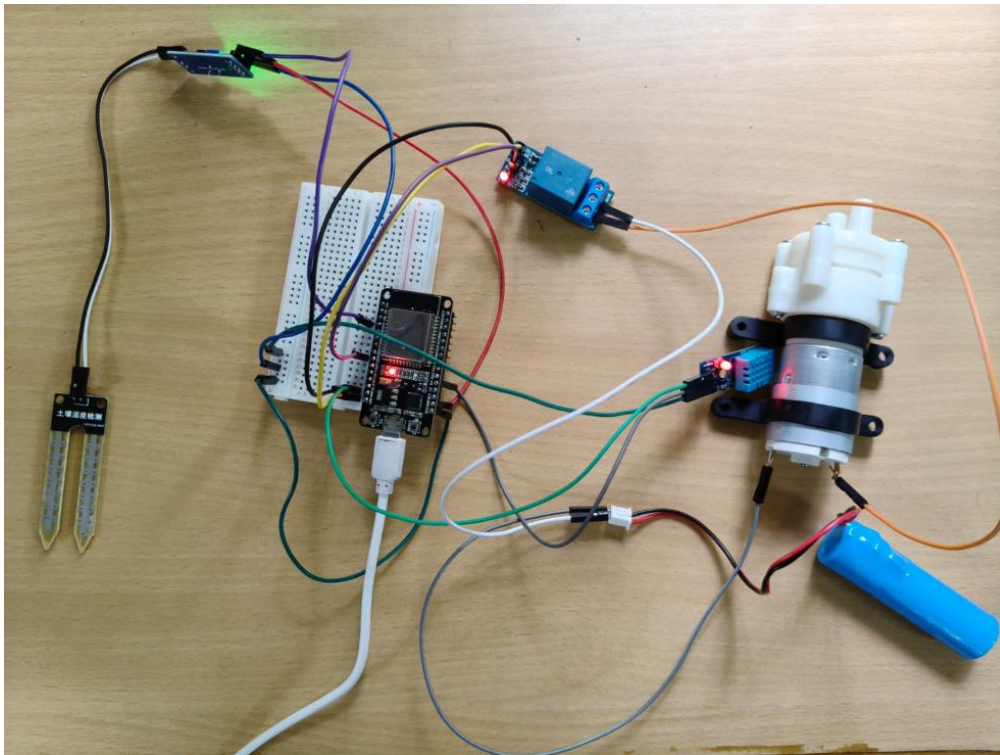
The process repeats continuously, creating an automatic closed-loop irrigation system.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component	Specification / Description	Quantity
1.	 ESP32 Development Board	ESP32 Wi-Fi + Bluetooth Development Board	1
2.	 Capacitive Soil Moisture Sensor	Soil Moisture Sensor (Analog Output)	1
3.	 DHT11 / DHT22 Sensor	Temperature & Humidity Sensor	1
4.	 Relay Module (1-Channel)	5V Relay Module for Pump Control	1
5.	 DC Water Pump	5V / 12V DC Submersible Water Pump	1
6.	 Power Supply	5V DC Power Supply (2A or above)	1
7.	 Breadboard	Solderless Breadboard	1
8.	 Jumper Wires	Male to Male / Male to Female Jumper Wires	As required
9.	 Water Pipe	PVC Pipe for Water Flow	As required
10.	 USB Cable	Micro USB Cable for Power & Programming	1
11.	 Wi-Fi Connection	Internet Connection for ESP32	1
12.	 Adafruit IO Cloud	Cloud Platform for Data Monitoring	1

11. Prototype Implementation:



The prototype is implemented using an ESP32 development board as the central controller.

The soil moisture sensor is inserted into the soil or a representative soil sample. The DHT11/DHT22 is positioned so that it can measure the surrounding temperature and humidity.

The water-level sensor is placed inside the water storage container. The relay module is connected to the ESP32 and controls the water pump.

The pump is connected to a water pipe directed toward the plant or crop area. The ESP32 receives sensor readings and automatically determines whether irrigation is necessary.

The IoT implementation, the ESP32 connects to a Wi-Fi network and sends sensor data to an online dashboard. The dashboard can display:

- Soil moisture.

- Temperature.
- Humidity.
- Water level.
- Pump status.

The prototype demonstrates the complete cycle from environmental sensing to automatic irrigation.

12. Results and Performance Analysis:

The Smart Irrigation and Water Management System was successfully designed and implemented using an ESP32, soil moisture sensor, DHT11/DHT22 sensor, relay module, and water pump. The system automatically monitors soil conditions and controls irrigation based on the moisture level.

Performance Analysis:

1. The soil moisture sensor continuously provides the moisture condition of the soil to the ESP32. When the moisture value falls below the predefined threshold, the system identifies the soil as dry and starts irrigation.
2. The relay module successfully controls the water pump according to the decision made by the ESP32. This eliminates the need for continuous manual operation of the pump.
3. The system supplies water only when the soil requires irrigation. Therefore, unnecessary watering is avoided, which helps improve water-use efficiency.
4. The DHT11/DHT22 sensor provides temperature and humidity readings. These parameters can be monitored along with soil moisture to understand the environmental conditions affecting the crop.

5. The ESP32 sends sensor readings to the Adafruit IO cloud using Wi-Fi. The dashboard allows the user to monitor soil moisture, temperature, humidity, and pump status remotely.

6. The ESP32 processes sensor readings quickly and responds automatically when the soil condition changes. Using separate ON and OFF thresholds also helps prevent frequent switching of the relay.

Overall Result:

The developed system demonstrates that IoT-based automatic irrigation can improve water management by supplying water according to the actual soil condition. The combination of ESP32, soil moisture sensing, automatic pump control, and Adafruit IO monitoring provides an efficient and user-friendly solution for smart agriculture.

The prototype can be further improved by adding weather forecasting, multiple irrigation zones, water-flow measurement, solar power, mobile notifications, and AI-based irrigation prediction.